

```
"""
```

AO* Search Implementation in Python

Statement: Implement the AO* search algorithm for traversing or searching tree or graph data structures.

```
"""
```

```
class AOSTar:
```

```
    def __init__(self, graph: dict, heuristic: dict):
```

```
        """
```

```
        Initialize the AO* search algorithm.
```

```
        Args:
```

```
            graph (dict): The graph represented as an adjacency list.
```

```
            heuristic (dict): The heuristic values for each node.
```

```
        """
```

```
        self.graph = graph
```

```
        self.heuristic = heuristic
```

```
        self.solution = {}
```

```
    def ao_star(self, node: str) -> float:
```

```
        """
```

```
        Perform AO* search starting from the given node.
```

```
        Args:
```

```
            node (str): The starting node for the search.
```

```
        Returns:
```

```
            float: The minimum cost to reach a goal node from the starting node.
```

```
        """
```

```
        # If the node is a leaf node (no children), return its heuristic value
```

```
        if node not in self.graph or not self.graph[node]:
```

```
            return self.heuristic[node] # Return heuristic value for leaf nodes
```

```
        min_cost = float('inf') # Initialize minimum cost to infinity
```

```
        best_child = None # Variable to store the best child node
```

```
        # Iterate through each group of children (AND/OR) for the current node
```

```
        for group in self.graph[node]:
```

```
            group_cost = 0 # Initialize cost for the current group
```

```
            for child, weight in group: # Iterate through each child in the group
```

```
                # Calculate total cost for the group (weight + cost of child)
```

```
                group_cost += weight + self.ao_star(child)
```

```
        # If the cost of this group is less than the current minimum cost
```

```
        if group_cost < min_cost:
```

```
            min_cost = group_cost # Update minimum cost
```

```
            best_child = group # Update best child group
```

```
        # Update heuristic value for the current node
```

```
        self.heuristic[node] = min_cost
```

```
        # Store the best child group in the solution dictionary
```

```
        self.solution[node] = best_child
```

```

    # Return the minimum cost to reach a goal node from the current node
    return min_cost

def print_solution(self, node: str):
    """
    Print the solution graph starting from the given node.

    Args:
        node (str): The starting node for the solution graph.
    """
    # If the node has a solution, print its children
    if node in self.solution:
        # Iterate through each child in the solution for the current node
        for child, _ in self.solution[node]:
            # Print the current node and its child
            print(f"{node} -> {child}")
            # Recursively print the solution for the child node
            self.print_solution(child)

def main():
    """
    Main function to demonstrate the AO* search algorithm.
    """
    # Sample graph with AND/OR relationships
    graph = {
        'A': [[('B', 1), ('C', 1)], [('D', 1)]], # A → (B AND C) OR D
        'B': [[('E', 1)], [('F', 1)]],
        'C': [[('G', 1)]],
        'D': [],
        'E': [],
        'F': [],
        'G': [],
    }

    # Heuristic values
    heuristic = {
        'A': 10,
        'B': 4,
        'C': 2,
        'D': 3,
        'E': 1,
        'F': 2,
        'G': 2
    }

    # Create an instance of AOStar and perform the search
    ao = AOStar(graph, heuristic)
    # Start AO* search from node 'A'
    ao.ao_star('A')

    print("Solution Graph:")
    # Print the solution graph starting from node 'A'
    ao.print_solution('A')

if __name__ == "__main__":

```

```
main()
```

```
# Output:
```

```
# Solution Graph:
```

```
# A -> D
```

ABHILASH B J